MySQL Notes: Sorting, Grouping, and Having Clause

1. Sorting Data

Sorting in MySQL is done using the `ORDER BY` clause.
The `LIMIT` clause is used to restrict the number of rows returned.

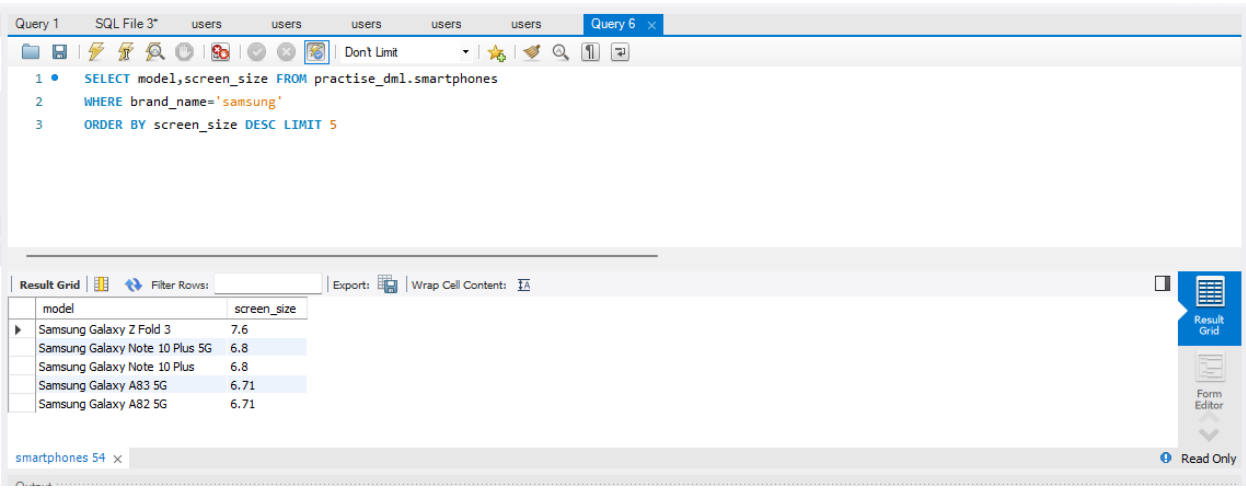
Basic Syntax

```
SELECT column1, column2
FROM table_name
ORDER BY column_name ASC|DESC
LIMIT n;
```

- `ASC` → Ascending order (default)
- `DESC` → Descending order

Examples

1. Top 5 Samsung phones with biggest screen size



2. Sort phones by total cameras (descending)

```sql
SELECT model,num_rear_cameras,num_front_cameras FROM practise_dml.smartphones
ORDER BY (num_rear_cameras + num_front_cameras)  DESC ;
```

model	num_rear_cameras	num_front_cameras
Vivo V21 Pro	4	2
Realme X50 Pro 5G (12GB RAM + 256GB)	4	2
OPPO F17 Pro	4	2
Motorola Edge S	4	2
Vivo V19 (8GB RAM + 256GB)	4	2
Realme 6 Pro	4	2
Tecno Phantom X Pro	4	2

3. Sort phones by PPI (decreasing order)

```sql
SELECT model,ROUND(SQRT((resolution_width*resolution_width)+(resolution_height*resolution_height))/screen_size,2) AS 'PPI'
FROM practise_dml.smartphones
ORDER BY PPI DESC ;
```

model	PPI
Sony Xperia 1 II	642.63
Samsung Galaxy S20	565.98
Samsung Galaxy S20 5G	565.98
Xiaomi 14	545.73
Google Pixel 2 XL	536.66
Sharp Aquos R5G	535.37
Samsung Galaxy S9 Plus (128GB)	530.92

4. Find phone with 2nd largest battery

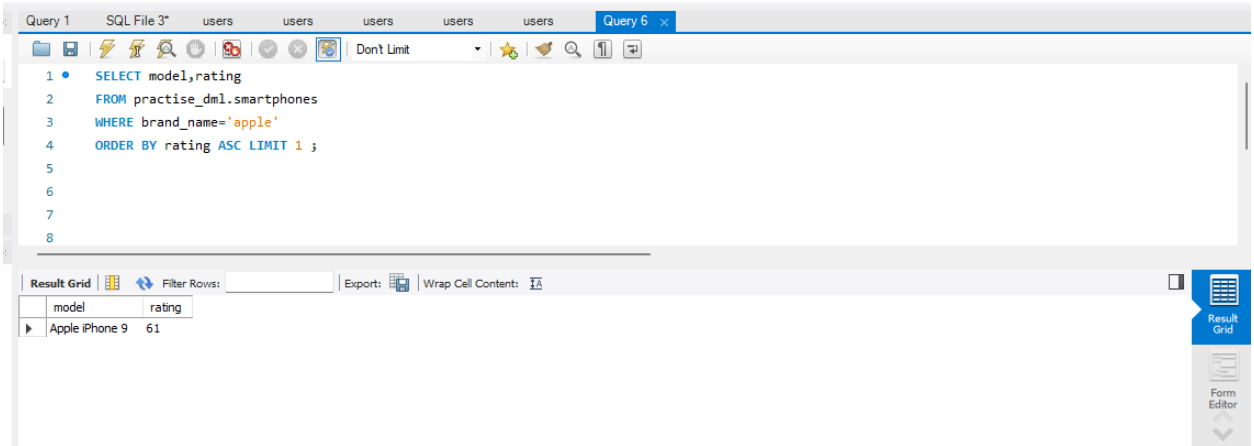
```sql
SELECT model,battery_capacity
FROM practise_dml.smartphones
ORDER BY battery_capacity DESC LIMIT 1,1 ;
```

model	battery_capacity
Oukitel WP19	21000

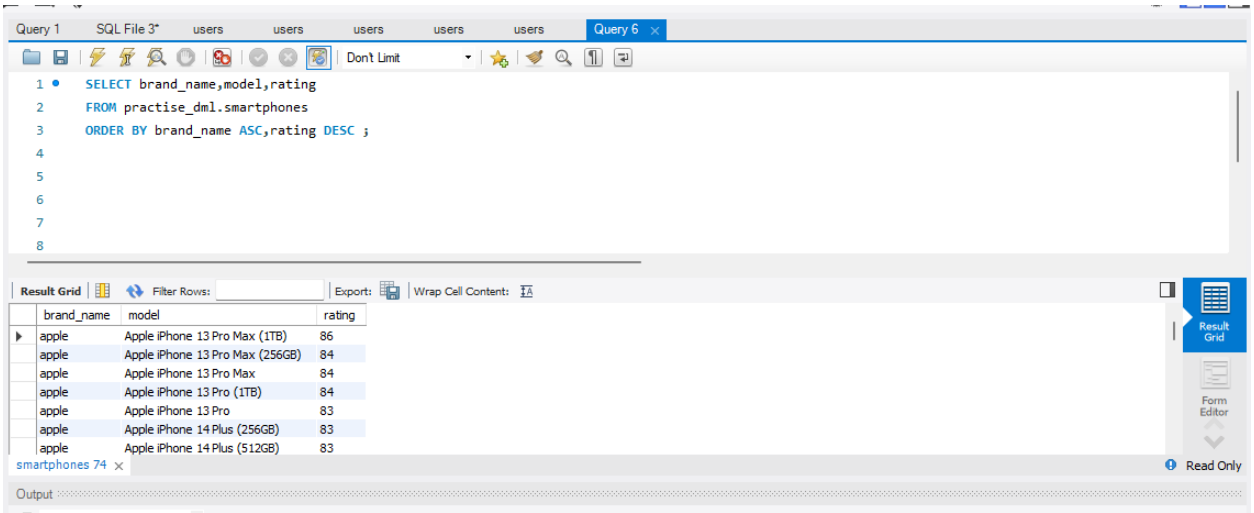
📌 Explanation:

- `LIMIT 1,1` → skip 1 row, then return 1 row
- Format:
- `LIMIT offset, count`

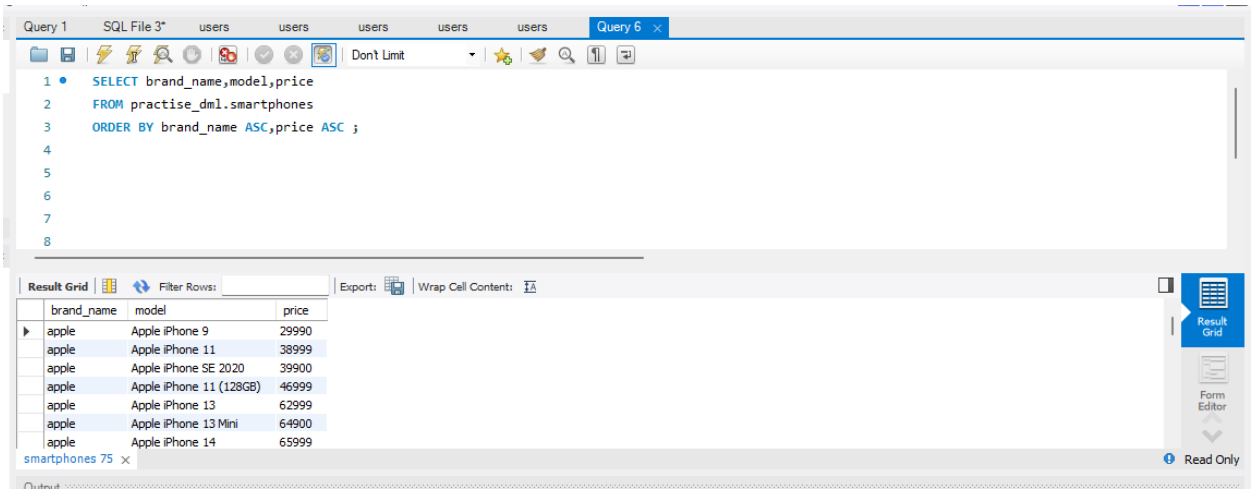
5. Worst rated Apple phone



6. Sort phones alphabetically, then by rating (desc)



7. Sort phones alphabetically, then by price (asc)



2. Grouping Data

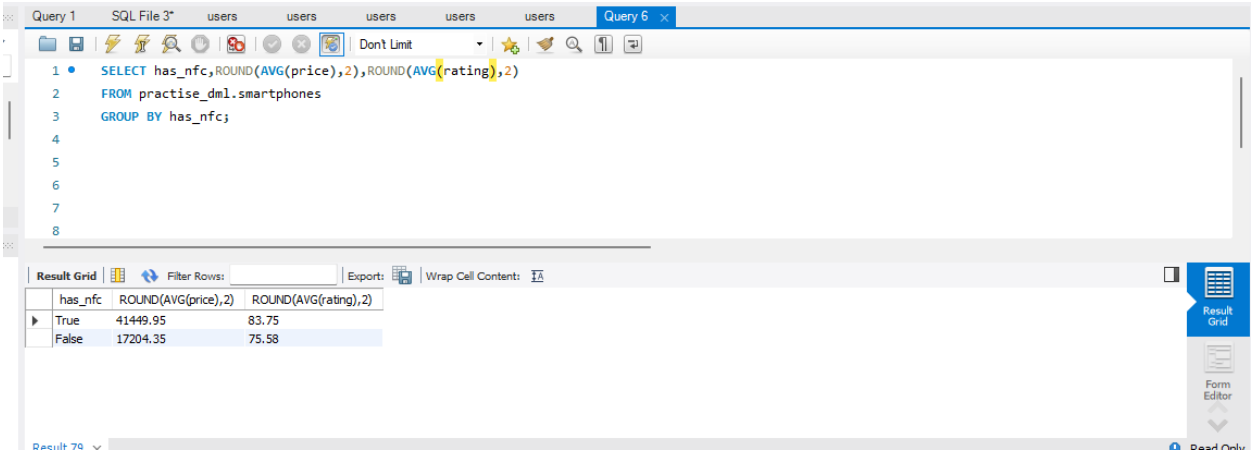
Grouping is done using the `GROUP BY` clause.
It groups rows based on column values and allows applying **aggregate functions** on each group.

Basic Syntax

```
SELECT column, AGG_FUNCTION(column)
FROM table_name
GROUP BY column;
```

Examples

1. Group by NFC and get avg price & rating



2. Group by extended memory and get avg price

```sql
SELECT extended_memory_available,ROUND(AVG(price),2)
FROM practise_dml.smartphones
GROUP BY extended_memory_available;
```

extended_memory_available	ROUND(AVG(price),2)
0	42961.23
1	17468.46

3. Group by brand & processor and get count + avg camera

```sql
SELECT brand_name,processor_brand,COUNT(model),ROUND(AVG(primary_camera_rear),2)
FROM practise_dml.smartphones
GROUP BY brand_name,processor_brand;
```

brand_name	processor_brand	COUNT(model)	ROUND(AVG(primary_camera_rear),2)
oneplus	snapdragon	27	52.48
samsung	exynos	42	44.24
motorola	snapdragon	33	60.85
realme	dimensity	27	53.59
samsung	snapdragon	24	46.08
apple	bionic	21	12
xiaomi	dimensity	36	81.47

4. Top 5 most costly phone brands

```sql
SELECT brand_name,MAX(price) AS "Highest_price"
FROM practise_dml.smartphones
GROUP BY brand_name
ORDER BY Highest_price DESC LIMIT 5
;
```

brand_name	Highest_price
apple	179900
huawei	162990
royole	149999
oppo	134999
leitz	124990

5. Brand with smallest screen smartphones

```sql
SELECT brand_name,MIN(screen_size) AS MINIMUM_SS
FROM practise_dml.smartphones
GROUP BY brand_name
ORDER BY MINIMUM_SS ASC LIMIT 1
;
```

brand_name	MINIMUM_SS
apple	4.7

6. Brand with most models having NFC + IR blaster

```sql
SELECT brand_name,COUNT(*) AS "no_of_model"
FROM practise_dml.smartphones
WHERE has_nfc='True' and has_ir_blaster='True'
GROUP BY brand_name
ORDER BY no_of_model DESC LIMIT 1
;
```

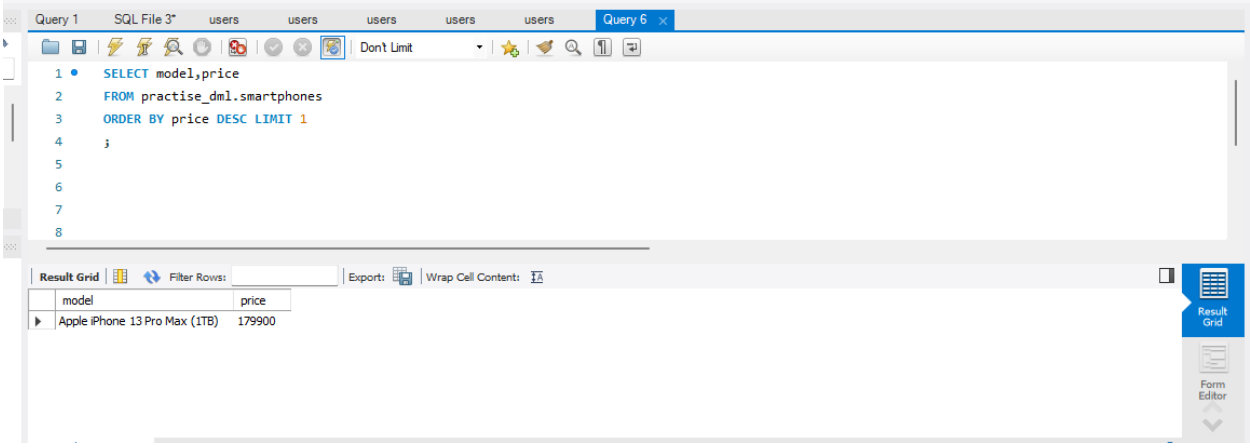
brand_name	no_of_model
xiaomi	27

7. Samsung 5G phones → avg price for NFC vs Non-NFC

```sql
SELECT has_nfc,AVG(price)
FROM practise_dml.smartphones
WHERE has_5g='True'
GROUP BY has_nfc
;
```

has_nfc	AVG(price)
True	43256.3927
False	23208.0980

8. Costliest phone (name + price)



3. HAVING Clause

The `HAVING` clause is used to filter results **after grouping**, especially when using aggregate functions.

Difference

- `WHERE` → filters rows **before grouping**
- `HAVING` → filters groups **after grouping**

Basic Syntax

```
SELECT column, AGG_FUNCTION(column)
FROM table_name
GROUP BY column
HAVING condition;
```

Examples

1. Avg rating of brands with more than 20 phones

```sql
SELECT brand_name,COUNT(*) AS 'no_of_phones',ROUND(AVG(rating),2) AS "Avg_rating"
FROM practise_dml.smartphones
GROUP BY brand_name
HAVING no_of_phones >20
;
```

brand_name	no_of_phones	Avg_rating
oneplus	37	82.41
samsung	103	79.46
motorola	48	80.1
realme	92	76.45
apple	21	79.19
xiaomi	124	78.7
oppo	82	78.73

2. Top 3 brands with highest avg RAM

Conditions:

- refresh rate ≥ 90
- fast charging available
- brands with less than10 phones

```sql
SELECT brand_name,COUNT(*) AS 'no_of_phones',ROUND(AVG(rating),2) AS "Avg_rating",ROUND(AVG(ram_capacity),2) AS "Avg_ram_capacity"
FROM practise_dml.smartphones
WHERE refresh_rate>=90 AND fast_charging_available=1
GROUP BY brand_name
HAVING no_of_phones <10
ORDER BY Avg_ram_capacity DESC LIMIT 3
;
```

brand_name	no_of_phones	Avg_rating	Avg_ram_capacity
lenovo	2	88	14
asus	6	87	13.33
leitz	1	89	12

3. Avg price of brands (rating > 70, phones > 10, only 5G phones)

Query 1 | SQL File 3* | users | users | users | users | users | Query 6

Dont Limit

```
SELECT brand_name,COUNT(*) AS 'no_of_phones',ROUND(AVG(rating),2) AS "Avg_rating",ROUND(AVG(price),2) AS "Avg_price"
FROM practise_dml.smartphones
WHERE has_5g="True"
GROUP BY brand_name
HAVING no_of_phones >10 AND Avg_rating >70
;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

brand_name	no_of_phones	Avg_rating	Avg_price
oneplus	31	84.06	34373.55
samsung	47	83.7	40019.55
motorola	28	84.18	28263.32
realme	51	80.96	23268.73
apple	16	82.12	102705.69
xiaomi	68	82.43	26714.37
oppo	47	82.66	31540.47

Result 117

Result Grid | Form Editor | Read Only